Smart Pointers and Ownership

Who owns an object: `unique_ptr`, `shared_ptr` and `weak_ptr`, the control block, cycles, and how to pass ownership around. Builds on RAII and Move Semantics.

Who owns it? The three smart pointers

RAII wrappers around `new` / `delete` (`<memory>`) that make **ownership explicit in the type**. The owner is whoever must free the object.

TYPE	OWNERSHIP	COPYABLE?	COST
`std::unique_ptr<T>`	exclusive: one owner	**no**, only movable	same as a raw pointer
`std::shared_ptr<T>`	shared: reference counted, freed when the last owner goes	yes (bumps the count)	extra control block, atomic count
`std::weak_ptr<T>`	none: observes a `shared_ptr` without keeping it alive	yes	small
raw pointer / reference	none: "I use it, I do not own it"	yes	none

DEFAULT A plain value member or local first. Then `unique_ptr` when it must live on the heap (polymorphism, large, optional). `shared_ptr` only for truly shared ownership.

A FUNCTION THAT...	TAKES
only uses the object	`const Widget&` (or `Widget*` if it may be null): never the smart pointer
takes over ownership	`std::unique_ptr<Widget>` by value: the caller must `std::move`

keeps a share	`std::shared_ptr<Widget>` by value
gives ownership to the caller	returns `std::unique_ptr<Widget>` (a factory)

```
std::unique_ptr<Widget> makeWidget(std::string n);        // factory: sole ownership to the caller
void use(const Widget& w);                                // just uses: pass the object, not the owner
void take(std::unique_ptr<Widget> w);                     // takes ownership

auto w = makeWidget("alpha");
use(*w);
take(std::move(w));                                       // w is now null
```

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 12.1 Dynamic memory and smart pointers (PDF pages 479 to 504)

Programming: Principles and Practice (Stroustrup)

- 18.4 to 18.5 Resources and exceptions; resource-management pointers (RAII, unique_ptr) (PDF pages 542 to 553)

C++ Move Semantics (Josuttis)

- Chapter 13 Move-only types (PDF pages 213 to 218)

`unique_ptr` in detail

```
auto p = std::make_unique<Widget>(1, "a");    // preferred over `new`
// auto q = p;                                // ERROR: unique_ptr cannot be copied
auto q = std::move(p);                        // OK: ownership moves, p is now null
```

STEP THROUGH **unique_ptr: ownership moves, it is never copied**

```
1  auto p = std::make_unique<Widget>();
2  auto q = std::move(p);
3  q.reset();
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `auto p = std::make_unique<Widget>();`	**p** owns the Widget. It is the only owner, and its destructor will free the object.	**`p = 0x5000 | Widget = alive (heap)`**
2	line 2 `auto q = std::move(p);`	`std::move(p)` hands ownership to **q**. p is now **null**. Copying (`auto q = p;`) does not compile: there can be only one owner.	**`p = nullptr | q = 0x5000`** `| Widget = alive (heap)`
3	line 3 `q.reset();`	`q.reset()` destroys the Widget now. Without it, the destructor of q would do the same at the closing brace. No `delete`, no leak, same cost as a raw pointer.	`p = nullptr |` **`q = nullptr`** ~~`Widget = destroyed (heap)`~~

- **Zero overhead:** with the default deleter `sizeof(unique_ptr<T>) == sizeof(T*)` (checked). A function-pointer deleter makes it twice as big; a stateless lambda deleter adds nothing.
- **Custom deleter** for anything that is not `delete`: `std::unique_ptr<std::FILE, decltype(closer)> f(std::fopen( ... ), closer);` closes the file on scope exit.
- **Arrays:** `std::make_unique<int[]>(3)` uses `delete[]`. Prefer `std::vector` unless you need a fixed block.
- **Polymorphism:** `unique_ptr<Base>` holding a `Derived` needs a **virtual destructor** on `Base`, as with any `delete` through a base pointer.
- `.get()` returns the raw pointer without giving up ownership; `.release()` gives it up (you must delete it); `.reset()` destroys the object now.

KEY POINT Why `make_unique`? No naked `new`, the type is written once, and before C++17 `f(unique_ptr<A>(new A), g())` could leak when `g()` threw between the two steps.

READ MORE

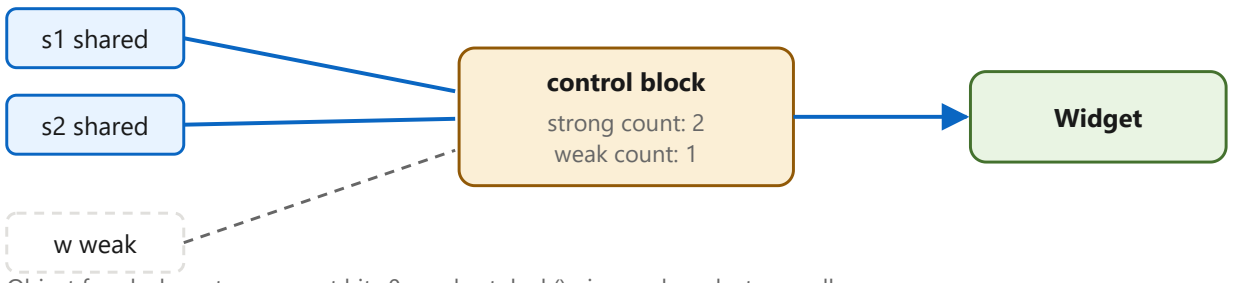
C++ Primer (Lippman, Lajoie, Moo)

- 12.1 Dynamic memory and smart pointers (PDF pages 479 to 504)

Effective Modern C++ (Meyers)

- Item 18, Use std::unique_ptr for exclusive-ownership resource management (PDF pages 136 to 143)
- Item 21, Prefer make_unique and make_shared to new (PDF pages 157 to 165)

`shared_ptr` and the control block



Object freed when strong count hits 0. weak_ptr.lock() gives a shared_ptr or null.

unique_ptr has no control block: zero overhead, one owner.

```
auto s1 = std::make_shared<Widget>(2, "b");  // one allocation for object + control block
auto s2 = s1;                                // two owners, use_count() == 2
```

STEP THROUGH **shared_ptr reference counting**

```
1  auto a = std::make_shared<Foo>();
2  auto b = a;
3  b.reset();
4  a.reset();
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `auto a = std::make_shared<Foo>();`	**make_shared** creates the Foo and its control block in one allocation. `a.use_count()` is 1.	**`a = 0x5000`** | **`Foo = 1 (heap)`**
2	line 2 `auto b = a;`	Copying a shared_ptr does **not** copy the Foo. It points to the same object and bumps the count to 2.	`a = 0x5000` | **`b = 0x5000`** | **`Foo = 2 (heap)`**
3	line 3 `b.reset();`	**b.reset()** drops one owner. The count falls to 1 and Foo is still alive because **a** owns it.	`a = 0x5000` | **`b = nullptr`** | **`Foo = 1 (heap)`**
4	line 4 `a.reset();`	The last owner is gone: the count hits 0 and **Foo is destroyed and its memory freed automatically**. If a and b had pointed at each other's owners (a cycle), this would never happen: use `weak_ptr`.	**`a = nullptr`** | `b = nullptr` ~~| `Foo = 0 (destroyed) (heap)`~~

- A `shared_ptr` is **two pointers**: to the object and to a **control block** holding the strong count, the weak count and the deleter.
- `make_shared` allocates object and control block together (one allocation). `shared_ptr<T>(new T)` allocates twice.
- The deleter is not part of the `shared_ptr` type (unlike `unique_ptr`): type-erased in the control block.
- The counts are **atomic**: copying or destroying `shared_ptr`s on several threads is safe. The **object** is not protected, and changing the same `shared_ptr` variable from two threads is a race.
- Copying costs an atomic increment. On hot paths pass `const shared_ptr&` or the object itself, not a copy per call.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 12.1 Dynamic memory and smart pointers (PDF pages 479 to 504)

Effective Modern C++ (Meyers)

- Item 19, Use std::shared_ptr for shared-ownership resource management (PDF pages 143 to 152)

Reference cycles and `weak_ptr`

If `A` holds a `shared_ptr<B>` and `B` holds a `shared_ptr<A>`, each keeps the other alive: **neither is ever destroyed**. Compiled: after the scope, both objects were still alive (leaked). Make the back link (child to parent) a `weak_ptr`: both were destroyed.

STEP THROUGH **A shared_ptr cycle leaks**

```
1  struct Node { shared_ptr<Node> next; };
2  {
3      auto a = make_shared<Node>();
4      auto b = make_shared<Node>();
5      a->next = b;
6      b->next = a;
7  }
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 3 `auto a = make_shared<Node>();`	**a** owns Node A. Strong count 1.	**a = 0x5000** | **Node A = null (1) (heap)**
2	line 4 `auto b = make_shared<Node>();`	**b** owns Node B. Strong count 1.	a = 0x5000 | **b = 0x6000** | Node A = null (1) (heap) | **Node B = null (1) (heap)**
3	line 5 `a->next = b;`	A now also owns B through `next`: the count of B is **2**.	a = 0x5000 | b = 0x6000 | **Node A = 0x6000 (1) (heap)** | **Node B = null (2) (heap)**
4	line 6 `b->next = a;`	B owns A back: the count of A is **2**. Each node keeps the other alive.	a = 0x5000 | b = 0x6000 | **Node A = 0x6000 (2) (heap)** | **Node B = 0x5000 (2) (heap)**
5	line 7 `}`	The scope ends and **a** and **b** are destroyed, but each count only falls to **1**: the other node still owns it. Nobody can reach them and nothing frees them: a **leak**. If `next` were a `weak_ptr` in one direction, the count of that node would stay 1, so it would be freed when its variable dies, which then releases the other.	~~a = gone | b = gone~~ | **Node A = 0x6000 (1) (heap)** | **Node B = 0x5000 (1) (heap)**

```
struct Node { std::shared_ptr<Owner> parent; };        // cycle: leak
struct WNode { std::weak_ptr<OwnerW> parent; };        // weak back-pointer:
freed

std::weak_ptr<Widget> w = s1;                          // observe only: does
not change use_count
if (auto sp = w.lock()) { /* still alive: use sp for as long as you hold it
*/ }   // lock() gives a shared_ptr or null
w.expired();                                           // true once every
owner is gone
```

- `lock()` is the only way to use the object: it gives a `shared_ptr` that keeps it alive while you work, or null.
- Typical uses: parent links, caches, observer lists that must not keep subscribers alive.
- A `weak_ptr` keeps the **control block** alive (not the object) until the last weak reference goes.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 12.1 Dynamic memory and smart pointers (PDF pages 479 to 504)

Effective Modern C++ (Meyers)

- Item 20, Use std::weak_ptr for shared_ptr-like pointers that can dangle (PDF pages 152 to 157)

Pitfalls and `enable_shared_from_this`

ONE CONTROL BLOCK PER OBJECT

```
auto raw = new Widget;
std::shared_ptr<Widget> a(raw);
std::shared_ptr<Widget> b(raw);        // a second, independent control
block: double delete!
```

Never build a second `shared_ptr` from a raw pointer or from `this`. Copy an existing one.

When a member function needs a `shared_ptr` to its own object (to register itself, to hand to a callback), inherit `std::enable_shared_from_this` and call `shared_from_this()`:

```
struct Session : std::enable_shared_from_this<Session> {
    std::shared_ptr<Session> self() { return shared_from_this(); }   // a
safe second owner, SAME control block
};
auto s = std::make_shared<Session>();
auto t = s->self();                    // use_count() == 2
Session raw; raw.self();               // throws std::bad_weak_ptr: no
shared_ptr owns it (compiled)
```

PITFALL	FIX
`shared_ptr` cycle	`weak_ptr` for the back link
`shared_ptr` from `this` or a raw pointer	`enable_shared_from_this`
`shared_ptr` by default "to be safe"	`unique_ptr`: you can always convert to shared later
storing `.get()` somewhere that outlives the owner	keep the smart pointer, or a `weak_ptr`
`delete` on a `.get()` result	never: the smart pointer will delete it again

Cheat sheet

Answer aloud, then open.

<details>
<summary>▼ <code>unique_ptr</code> vs <code>shared_ptr</code> vs <code>weak_ptr</code>?</summary>

<code>unique_ptr</code>: sole owner, move-only, zero overhead. <code>shared_ptr</code>: reference-counted shared ownership with a control block and atomic count. <code>weak_ptr</code>: non-owning observer used to break cycles and to check whether a shared object is still alive (<code>lock()</code>).
</details>

<details>
<summary>▼ Is <code>shared_ptr</code> thread-safe?</summary>

The reference count is atomic, so copying and destroying <code>shared_ptr</code>s from different threads is safe. The pointed-to object is **not** protected: concurrent access to it still needs synchronization. Concurrently modifying the same <code>shared_ptr</code> variable is also a race.
</details>

<details>
<summary>▼ Why prefer <code>make_unique</code> / <code>make_shared</code>?</summary>

No naked <code>new</code>, less typing (and exception safe before C++17), and <code>make_shared</code> allocates the object and its control block once instead of twice.
</details>

<details>
<summary>▼ How do you break a <code>shared_ptr</code> cycle?</summary>

Make one direction (usually the child's link to its parent) a <code>weak_ptr</code>. It does not count toward ownership, so the objects can be destroyed; use <code>lock()</code> to get a temporary <code>shared_ptr</code> when needed.
</details>

▼ How do you pass a smart pointer to a function?

If the function only uses the object, pass a reference or raw pointer, not the smart pointer. To transfer ownership, take a `unique_ptr` by value and have the caller `std::move`. To share, take a `shared_ptr` by value. Return `unique_ptr` from factories.

▼ Why can two `shared_ptr`s built from the same raw pointer crash?

Each creates its own control block and count, so the object is deleted twice. Copy an existing `shared_ptr`, or inherit `enable_shared_from_this` when an object must hand out a shared pointer to itself.

▼ Does `unique_ptr` cost anything compared with a raw pointer?

Not with the default deleter: it is the same size and the destructor inlines to a `delete`. A stateful deleter (a capturing lambda, a function pointer) makes it larger.